

AI Visibility Tool

Project Context and Status

Faculty Assistance in Data Science (FADS)

Advisor: Prof. Nick Brown

Prepared by: Manas Dani

This document is a living project memory. It can be shared at the start of any future discussion to re-establish full context quickly.

1. Project Overview

This project is part of a Faculty Assistance in Data Science (FADS) role under Prof. Nick Brown. The broader vision is to support theLLMStore, a future marketplace where AI-related assets can be shared and reused. While the long-term idea includes personal or local LLMs, the current and near-term focus is on building a practical, usable data extraction and cleaning tool.

The AI Visibility Tool is being redesigned as an intelligence extraction layer. Instead of directly uploading or sharing full LLM models (which presents platform, hosting, and privacy challenges), the tool focuses on extracting, structuring, and packaging high-quality knowledge from user-provided content so it can be reused across many AI systems.

2. Core Idea (Agreed Direction)

The AI Visibility Tool will:

- Accept user-provided content (initially websites, later PDFs, documents, blogs)
- Extract raw content and readable text
- Clean, structure, and organize the information
- Produce portable intelligence artifacts (clean text, chunks, metadata, schema)
- Enable reuse of these artifacts by:
 - Local LLM tools (e.g., AnythingLLM)
 - RAG-based applications
 - Future sharing via theLLMStore

theLLMStore, in this framing, becomes a place to store and share intelligence artifacts, not necessarily full LLM models.

3. What Has Been Completed

3.1 Conceptual and Design Work

- Clear understanding of the existing AI Visibility Tool codebase
- Identification of key shortcomings in the current prototype:
 - Tightly coupled crawling, scoring, and suggestions
 - No reusable or exportable output
 - Arbitrary and qualitative scoring without prioritization
 - Generic LLM suggestions

- Proposal of a redesigned, modular workflow

3.2 Workflow Design

A complete end-to-end workflow diagram has been designed and approved. The pipeline consists of nine stages:

Stage	Name	Description
1	User Input	Accept content from users (URLs, documents)
2	Input Validation	Verify and validate provided inputs
3	Source Collection	Crawl/ingest and store raw content
4	Cleaning & Normalization	Remove noise, normalize structure
5	Chunking & Labeling	Segment content into reusable units
6	Metadata & Schema Generation	Enrich chunks with metadata
7	Artifact Packaging	Package into portable intelligence artifacts
8	Storage	Store artifacts locally for reuse
9	Reuse (External)	Enable RAG, local LLM, and marketplace use

This workflow diagram was reviewed by Prof. Nick Brown and explicitly approved as the correct direction.

4. Scope Alignment

In Scope	Out of Scope (for now)
✓ User workflow	✗ Full theLLMStore marketplace UX
✓ Extraction and cleaning pipeline	✗ Payments, pricing, or licensing
✓ Data outputs and artifact formats	✗ Large-scale or production-grade crawling
✓ Reuse for AI visibility, RAG, and local LLMs	✗ Automated model training or fine-tuning
	✗ Hosting or serving user LLMs
	✗ Production authentication or infra

5. Current Stage and Status

The project moved into Week 3 with an agreed starting point at Stage 3: Source Collection. Prof. Nick Brown explicitly approved beginning at this stage. Stages 1 and 2 (User Input and Input

Validation) already exist for website URLs and will only require minor tweaks later to support document uploads.

6. Stage 3: Source Collection (Definition)

Stage 3 is strictly about collecting and storing raw sources.

6.1 What Stage 3 Does

- Opens the user-provided website or document
- Collects raw content (HTML or file content)
- Collects readable text
- Captures basic metadata (URL or filename, title, retrieval time)
- Stores information for reuse in later stages

6.2 What Stage 3 Does NOT Do

- No cleaning
- No chunking
- No scoring
- No embeddings
- No summaries
- No LLM-based suggestions

Stage 3 ends once raw content and metadata are safely collected and stored. It enables a crawl-once, analyze-many approach.

6.3 Why Stage 3 Matters

Stage 3 enables reusability: crawl or ingest once, then analyze, clean, score, and reuse many times. Without this stage being cleanly designed, later steps like artifact packaging, dashboards, and AnythingLLM integration are not possible.

7. Week 3 Implementation Update

During Week 3, Stage 3 (Source Collection) was not only designed but also implemented as a working proof of concept. A local prototype was built that allows users to ingest personal documents and extract raw, reusable content entirely on their own device. This aligns directly with the LLM

Bundler concept discussed with Prof. Nick Brown, where users maintain full ownership and control of their data.

7.1 Supported File Types

- PDF files
- Word documents (DOCX)
- PowerPoint presentations (PPTX)
- Plain text inputs

7.2 Processing per Source

- File ingestion on the local machine
- Raw text extraction
- Basic metadata capture (filename, file type, extraction timestamp, page/slide count where applicable)
- Local storage of extracted outputs for reuse in later stages

No cleaning, chunking, embeddings, scoring, or LLM-based transformations are performed at this stage, by design.

7.3 Validation and Approval

A recorded walkthrough video (~6 minutes) was created to demonstrate ingestion of multiple document types, extraction of raw text and metadata, local storage behavior, and alignment with the proposed workflow diagram. Prof. Nick Brown reviewed the implementation and explicitly confirmed that the proof of concept matches the intended vision and serves as a strong foundation for future stages.

7.4 Business Model Alignment (Prosumer Framing)

The implemented Stage 3 prototype directly supports the prosumer-based business model for theLLMStore:

- Users download and run the extraction tool locally
- Users choose which personal files to process
- All raw content extraction occurs on the user's device
- Users retain full copyright and intellectual property ownership
- theLLMStore does not host user data or LLMs, acting instead as a market maker and discovery layer

The extracted outputs are intended to later be transformed into AI-agent-accessible formats while preserving user ownership and minimizing legal and infrastructure risks.

8. Week 4: Cleaning, Chunking, and Local RAG Validation

8.1 Text Cleaning Strategy

The extracted text is processed using a light cleaning approach that removes obvious noise while preserving meaningful structure that large language models rely on. This includes normalizing excessive whitespace, fixing PDF-related hyphenation issues, and removing non-printable characters, while intentionally keeping paragraph breaks, headings, lists, and punctuation intact.

8.2 Hybrid Chunking Approach

After cleaning, the text is divided into reusable chunks using a hybrid chunking strategy:

- Content is first split along semantic boundaries (paragraphs, sections)
- Only further divided by sentences or fixed-size limits if necessary
- Target chunk size: 500–800 characters, with a hard maximum of 1,200 characters
- Small overlap between adjacent chunks preserves context across boundaries

This layered approach ensures that chunks remain meaningful, reusable, and optimized for retrieval in RAG pipelines and local LLM systems.

8.3 Chunk ID Generation

Each chunk is assigned a deterministic, human-readable ID by combining a stable hash of the document (derived from the filename and a portion of its content) with an ordered chunk index. This ensures reproducibility across runs, uniqueness across documents, and safe reuse in downstream workflows such as embeddings, vector storage, and RAG pipelines.

8.4 Local RAG Proof of Concept

The goal of this phase was to validate that preprocessed, locally controlled document artifacts can be embedded and queried through a local LLM pipeline, demonstrating the feasibility of a permissioned, personalized knowledge layer.

Work Completed

1. **Chunk Utilization:** Document chunks generated during Stage 4 were used as foundational knowledge units, preserving deterministic identifiers, controlled text boundaries, and consistent metadata.
2. **Vector Embedding Generation:** Chunked artifacts were ingested into AnythingLLM, where vector embeddings were generated locally, confirming compatibility with external RAG tooling.
3. **Local Knowledge Base Construction:** A dedicated workspace was created within AnythingLLM as a localized vector index, with all embeddings stored locally.
4. **Local LLM Integration and Querying:** A local LLM was connected to the embedded knowledge base, enabling question answering over document content with responses grounded in the ingested chunks.

5. **Validation of Permissioned Retrieval:** The system demonstrated that answers were generated from explicitly ingested, user-approved content rather than uncontrolled external sources.

Key Outcome

Preprocessing documents into structured, deterministic chunks prior to embedding provides a meaningful advantage over direct file uploads. The approach improves transparency, reproducibility, and explainability in retrieval-based LLM systems while preserving local ownership and privacy of user data.

9. Current Status Summary

- Stage 3 (Source Collection) — Complete and validated
- Stage 4 (Cleaning and Normalization) — Complete with proof of concept
- End-to-end pipeline operational: local ingestion → controlled chunking → local embedding → grounded LLM Q&A
- Project is ready to advance to subsequent stages (metadata enrichment, artifact packaging, RAG integration)

10. Long-Term Goal

To evolve the AI Visibility Tool into:

- A reusable intelligence extraction system
- A foundation for AI visibility optimization
- A contributor to theLLMStore ecosystem through shareable intelligence artifacts